

— API REST · V1

API de reducción de inventario

Guía de integración para descontar stock en la bodega de una tienda autorizada, mediante una API key operativa creada en el panel del sistema.

POST /managed/inventory/reduce-by-sku**POST** /managed/inventory/reduce-by-id

VERSIÓN
1.0

ACTUALIZADO
Mayo 2026

AUDIENCIA
**Desarrolladores
externos**

AUTENTICACIÓN
X-API-Key

UNA SOLUCIÓN DE
dot:solutions

00 TABLA DE CONTENIDOS

Contenido

El manual sigue un recorrido lineal: qué resuelve la API, cómo se configura, cómo se invoca y qué hacer cuando algo falla.

01	Introducción	qué resuelve y casos de uso	03
02	Antes de empezar	requisitos previos	04
03	Conceptos clave	tenant, autorización, IDs	05
04	Autenticación	headers y flujo de validación	07
05	Referencia de endpoints	reduce-by-sku y reduce-by-id	08
06	Idempotencia	el campo reference	10
07	Validaciones y códigos de error	qué valida el servidor	11
08	Buenas prácticas	seguridad, diseño, reintentos	13
09	Ejemplos completos	cURL listos para usar	14
10	Soporte	cómo obtener IDs y ayuda	16
	INTEGRACIÓN EXTERNA		02

01 INTRODUCCIÓN

Qué resuelve esta API

Permite **descontar stock** en la bodega asociada a una tienda autorizada en tu API key. Cada operación registra movimientos de inventario con trazabilidad completa.

Cada reducción genera un movimiento con el tipo indicado por ítem (por defecto **Consumo**) y documento `API_INVENTORY_REDUCTION`. La **bodega no se envía en el cuerpo** de la petición: el servidor la resuelve automáticamente según el emparejamiento tienda ↔ bodega configurado en tu key.

Dos formas de identificar productos

ENDPOINT	IDENTIFICACIÓN DE PRODUCTO
POST reduce-by-sku	Por SKU dentro de la tienda
POST reduce-by-id	Por ID externo del producto (<code>product.external_id</code> en la tienda)

Casos de uso típicos

PUNTO DE VENTA (POS)

Al cerrar un ticket, descuenta los productos vendidos usando el SKU del catálogo.

E-COMMERCE

Cada pedido confirmado reduce stock con el número de orden como `reference`.

ERP / SISTEMAS PROPIOS

Sincroniza consumos y mermas mapeando tus IDs vía `external_id`.

PRODUCCIÓN / MERMAS

Registra consumo interno o desperdicio clasificado por `movement_type`.

EN UNA FRASE

Tú controlas **la tienda y los productos**; el servidor decide la bodega y registra el movimiento de forma auditable.

02 ANTES DE EMPEZAR

Requisitos previos

Antes de llamar a los endpoints, tu organización debe tener configurado lo siguiente en el panel del sistema.

- 1 API key de tipo **OPERATIVE** — una key **GENERAL** no sirve para estas rutas.
- 2 El módulo **inventory_reduction** asignado a esa key.
- 3 Al menos un par **tienda + bodega** autorizado
- 4 Productos dados de alta en la tienda, **con stock** en la bodega configurada.
- 5 **external_id** configurado en tiendas y productos que uses en la integración.

GUARDA LA KEY AL CREARLA

La key en texto plano **solo se muestra una vez** al crearla en el dashboard. Guárdala de forma segura; no se puede recuperar después.

Lo que controlas tú vs. lo que resuelve el servidor

TÚ ENVÍAS	EL SERVIDOR RESUELVE
Tienda (store_id , ID externo)	Bodega asociada a esa tienda en la key
Productos (SKU o product_id)	Producto en tienda y su stock disponible
Cantidad y movement_type	Tipo de movimiento y documento de registro
reference (opcional)	Idempotencia y trazabilidad del movimiento

03 CONCEPTOS CLAVE

URL base y tenant

El tenant se resuelve por el **subdominio** del host de la petición. Todas las rutas viven bajo el prefijo de la API v1.

— ESTRUCTURA DE LA URL

```
ht t ps: // { s ubdomi ni o} . { domi ni o} / api / v1 / managed / i nvent or y / . . .
```

Por ejemplo, `mi - empresa . t u - saas . com` resuelve al tenant `mi - empresa`.

ENTORNO	EJEMPLO DE HOST
Producción	<code>mi - empresa . t u - saas . com</code>

03 CONCEPTOS CLAVE

Identificadores

Todos los IDs que envías en el body de estos endpoints son **IDs externos** de tu sistema, mapeados en Smart Order mediante `external_id`.

Tienda

CAMPO	ORIGEN EN SMART ORDER	DESCRIPCIÓN
<code>store_id</code>	<code>store.external_id</code>	Identificador de tienda en tu ERP/POS

Producto (solo en reduce-by-id)

CAMPO	ORIGEN EN SMART ORDER	DESCRIPCIÓN
<code>product_id</code>	<code>product.external_id</code>	ID del producto maestro, resuelto en la tienda del body

Coordina con el administrador de tu instancia para que configure el `external_id` de las tiendas y productos que vas a usar en la integración.

SKU (REDUCE-BY-SKU)

El SKU debe existir en la tienda indicada. Es la opción más simple si tu catálogo ya usa SKU en Smart Order.

04 AUTENTICACIÓN

Headers y flujo de validación

Incluye la API key en **cada** petición. El servidor valida tenant, key, tipo y módulo antes de tocar el inventario.

HEADER	VALOR
X-API-Key	Tu API key completa (texto plano emitido al crearla)
Content-Type	application/json

Flujo de validación (en orden)



CONDICIÓN	HTTP	MENSAJE TÍPICO
Sin header X-API-Key	401	Unauthorized
Key inválida, revocada o inexistente	401	Unauthorized
Key no es OPERATIVE	403	Esta ruta requiere una API key operativa

CONDICIÓN**HTTP****MENSAJE TÍPICO**Key sin módulo `inventory_reduction`**403**

La API key no tiene permiso para reducción de inventario

05

REFERENCIA DE ENDPOINTS

Reducir por SKU

Descuenta inventario identificando cada ítem por su **SKU** dentro de la tienda indicada.

POST

/api/v1/managed/inventory/reduce-by-sku

Reducir por SKU

CAMPO	TIPO	REQ.	DESCRIPCIÓN
store_id	string	sí	ID externo de la tienda (store.external_id)
reference	string	NO	Referencia para idempotencia
items	array	sí	Lista de productos a descontar (mínimo 1)
items[].sku	string	sí	SKU del producto en la tienda
items[].quantity	number	sí	Cantidad a descontar; debe ser > 0
items[].movement_type	string	NO	withdrawal (Consumo, def.) · sale (Venta) · waste (Merma)

EJEMPLO DE BODY

```

{
  "store_id": "TIENDA-CENTRO-01",
  "reference": "venta-pos-2026-05-15-0042",
  "items": [
    { "sku": "PROD-001", "quantity": 2.5, "movement_type": "sale" },
    { "sku": "PROD-002", "quantity": 1 }
  ]
}
```

Tipos de movimiento

VALOR	REGISTRO	CUÁNDO USARLO
withdrawal	Consumo (por defecto)	Consumo interno o salida genérica
sale	Venta	Descuento por venta (POS / e-commerce)

VALOR	REGISTRO	CUÁNDO USARLO
waste	Merma	Desperdicio, caducidad, pérdida

05 REFERENCIA DE ENDPOINTS

Reducir por ID

Descuenta por **ID externo del producto maestro** (`product.external_id`) en la tienda indicada.

POST

`/api/v1/managed/inventory/reduce-by-id`

Reducir por ID externo de producto

CAMPO	TIPO	REQ.	DESCRIPCIÓN
<code>store_id</code>	string	SI	ID externo de la tienda (<code>store.external_id</code>)
<code>reference</code>	string	NO	Referencia para idempotencia
<code>items</code>	array	SI	Lista de productos (mínimo 1)
<code>items[].product_id</code>	string	SI	<code>product.external_id</code> en esa tienda
<code>items[].quantity</code>	number	SI	Cantidad a descontar; debe ser <code>> 0</code>
<code>items[].movement_type</code>	string	NO	<code>withdrawal</code> · <code>sale</code> · <code>waste</code>

— EJEMPLO DE BODY

```
{
  "store_id": "TIENDA-CENTRO-01",
  "reference": "ecommerce-order-98765",
  "items": [
    {
      "product_id": "ERP-PROD-5544",
      "quantity": 3,
      "movement_type": "sale"
    },
    {
      "product_id": "ERP-PROD-7788",
      "quantity": 0.5,
      "movement_type": "waste"
    }
  ]
}
```

— RESPUESTA EXITOSA · AMBOS ENDPOINTS

```
HTTP 200 OK
{
```

```
"ok": true  
}
```

06 IDEMPOTENCIA

El campo `reference`

El campo opcional `reference` evita descontar stock dos veces si reenvías la misma operación tras un timeout o un reintento de red.

REFERENCE EN EL BODY	COMPORTAMIENTO
Vacío u omitido	Se genera un UUID interno. Cada llamada descuenta stock (no idempotente).
Valor no vacío	Antes de descontar, se busca un movimiento previo con la misma combinación. Si ya existe → 200 con <code>{"ok": true}</code> sin volver a descontar .

Alcance de la clave de idempotencia

La clave es única por la combinación de tres elementos:

— CLAVE

```
( reference , API_INVENTORY_REDUCTION , api-key:<uuid> )
```

Es decir: el valor de `reference`, el tipo de documento `API_INVENTORY_REDUCTION` y la API key que realiza la operación. Dos keys distintas **pueden** usar la misma `reference` sin conflicto entre sí.

RECOMENDACIÓN

Usa siempre `reference` con un ID estable de tu sistema (número de pedido, UUID de transacción). Si recibes **200** tras un reintento con la misma `reference`, puedes asumir que la reducción ya se aplicó.

— PATRÓN RECOMENDADO

```
{
  "reference": "mi-sistema:orden-12345",
  "store_id": "...",
  "items": [ /* ... */ ]
}
```

07 VALIDACIONES Y ERRORES

Qué valida el servidor

Además de autenticación y permisos, el servidor valida los datos de cada ítem antes de descontar. Todas estas devuelven **400**.

VALIDACIÓN	MENSAJE TÍPICO
SKU vacío	SKU vacío en ítems
Cantidad ≤ 0	La cantidad debe ser mayor a 0 para el SKU ...
<code>movement_type</code> inválido	400 con <code>param</code> <code>movement_type</code>
Producto / SKU no existe en la tienda	El producto con SKU ... no existe en la tienda
Stock insuficiente	Stock insuficiente ... Disponible: X, solicitado: Y
<code>store_id</code> desconocido	No se encontró tienda con el <code>store_id</code> indicado
<code>product_id</code> desconocido	No existe producto con <code>product_id</code> ... en la tienda
JSON inválido / campos faltantes	400 con <code>param</code> del campo

CADA MOVIMIENTO QUEDA REGISTRADO CON

Tipo: según `movement_type` · **Documento:** `API_INVENTORY_REDUCTION` · **Observación:** incluye el nombre de la API key para trazabilidad.

07 VALIDACIONES Y ERRORES

Códigos de error

Las respuestas de error son un **array JSON**. El campo `param` solo aparece en errores de validación de entrada (**400**).

— FORMA DE LA RESPUESTA DE ERROR

```
[
  {
    "message": "Descripción del error",
    "param": "nombre_campo"
  }
]
```

Tabla de códigos HTTP

CÓDIGO	CUÁNDO
200	Operación exitosa (o idempotente ya aplicada)
400	Body inválido, reglas de negocio, producto no encontrado, stock insuficiente
401	Sin API key, key inválida, tenant no especificado o no encontrado
403	Key no operativa, sin módulo, tienda no autorizada, tenant inactivo
500	Error interno (p. ej. fallo al resolver tenant)

Mensajes frecuentes de validación (400, con param)

MENSAJE	PARAM
<code>store_id</code> requerido (binding)	<code>store_id</code>
<code>product_id</code> requerido (binding)	<code>product_id</code>

```
[
  {
    "message": "store_id requerido",
    "param": "store_id"
  }
]
```

```
[
  {
    "message": "La tienda no está
    autorizada para esta API key"
  }
]
```


08 BUENAS PRÁCTICAS

Recomendaciones de integración

Pautas para una integración segura, eficiente y resiliente a fallos transitorios.

Seguridad

- **Nunca** expongas la API key en el frontend ni en repositorios públicos.
- Rota o desactiva keys comprometidas desde el dashboard.
- Usa **HTTPS** en producción.

Diseño de la integración

1. **Siempre envía** `reference`, salvo que cada llamada deba ser independiente a propósito.
2. **Agrupar ítems en una sola petición** cuando una operación afecta varios productos: menos latencia, un solo documento de referencia.
3. **Elige un endpoint y mantén coherencia**: catálogo por SKU → `reduce-by-sku`; IDs de ERP → `reduce-by-id` con `product_id`.
5. **Mapea las tiendas autorizadas** en tu sistema con los `external_id` configurados en Smart Order y asociados a la key.

Manejo de errores y reintentos

SITUACIÓN	ACCIÓN
401	Revisar key y host (subdominio). No reintentar sin corregir credenciales.
403	Revisar tipo de key, módulo y tienda en el dashboard.
400 stock insuficiente	No reintentar con los mismos datos; sincronizar stock o ajustar cantidades.
400 producto no encontrado	Verificar SKU/IDs y tienda antes de reintentar.
500 / timeout	Reintentar con la misma <code>reference</code> para evitar doble descuento.
200 tras reintento	Tratar como éxito (idempotencia).

REINTENTOS

El servidor reintentará internamente hasta 3 veces ante conflictos. Aun así, conviene que tu cliente reintente los `5xx` con **backoff exponencial**.

09 EJEMPLOS COMPLETOS

Ejemplos con cURL

Sustituye `{subdominio}`, `{dominio}` y `{API_KEY}` por los valores de tu entorno, y los IDs externos y SKU por valores reales.

— REDUCIR POR SKU · VARIOS ÍTEMS

```
curl -X POST "https://{subdominio}.{dominio}/api/v1/managed/inventory/reduce-by-sku" \
-H "X-API-Key: {API_KEY}" \
-H "Content-Type: application/json" \
-d '{
  "store_id": "SUCURSAL-NORTE",
  "reference": "pos-ticket-10042",
  "items": [
    { "sku": "BEB-001", "quantity": 2, "movement_type": "sale" },
    { "sku": "SNK-010", "quantity": 1.5 }
  ]
}'
```

— REDUCIR POR SKU · UN ÍTEM

```
curl -X POST "https://{subdominio}.{dominio}/api/v1/managed/inventory/reduce-by-sku" \
-H "X-API-Key: {API_KEY}" \
-H "Content-Type: application/json" \
-d '{
  "store_id": "SUCURSAL-NORTE",
  "reference": "pos-ticket-10043",
  "items": [
    { "sku": "BEB-001", "quantity": 1 }
  ]
}'
```

09 EJEMPLOS COMPLETOS

Ejemplos con cURL (continuación)

— REDUCIR POR ID DE PRODUCTO

```
curl -X POST "https://{subdominio}.{dominio}/api/v1/managed/inventory/reduce-by-id" \
-H "X-API-Key: {API_KEY}" \
-H "Content-Type: application/json" \
-d '{
  "store_id": "SUCURSAL-NORTE",
  "reference": "woocommerce-order-98765",
  "items": [
    {
      "product_id": "ERP-PROD-5544",
      "quantity": 3,
      "movement_type": "sale"
    }
  ]
}'
```

— REDUCIR POR ID · MERMA

```
curl -X POST "https://{subdominio}.{dominio}/api/v1/managed/inventory/reduce-by-id" \
-H "X-API-Key: {API_KEY}" \
-H "Content-Type: application/json" \
-d '{
  "store_id": "SUCURSAL-NORTE",
  "reference": "erp-mov-2026-0515-001",
  "items": [
    {
      "product_id": "ERP-SKU-9988",
      "quantity": 5,
      "movement_type": "waste"
    }
  ]
}'
```

— RESPUESTA EXITOSA

```
"ok": true // HTTP 200 - { "ok": true }
```

Checklist de pruebas

- 1 Crear una API key **OPERATIVE** con módulo **inventory_reduction** y una tienda de prueba.

2 Reducir con una `reference` fija **dos veces** → la segunda debe devolver `200` sin cambiar stock adicional.

3 Probar con una tienda no autorizada → `403` .

4 Probar cantidad mayor al stock → `400` con mensaje de stock disponible.

10 SOPORTE

¿Necesitas ayuda?

Para configurar `external_id` en tiendas y productos, coordina con el administrador de tu instancia Smart Order o consulta la documentación interna del panel de API keys.

OBTENER IDENTIFICADORES

El administrador configura los `external_id` en tiendas y productos para que uses tus propios IDs en el body de las peticiones.

CONFIGURAR LA KEY

El emparejamiento tienda ↔ bodega, el tipo `OPERATIVE` y el módulo `inventory_reduction` se gestionan en el panel de API keys.

Resumen de un vistazo

TEMA	CLAVE
Endpoints	POST <code>/managed/inventory/reduce-by-sku</code> · <code>reduce-by-id</code>
Autenticación	Header <code>X-API-Key</code> + host con subdominio de tenant
Requisito de key	Tipo <code>OPERATIVE</code> + módulo <code>inventory_reduction</code>
Bodega	La resuelve el servidor; nunca se envía en el body
Idempotencia	Envía <code>reference</code> estable por operación
Éxito	200 <code>{ "ok": true }</code>